

Pydantic for GenAI & Agentic AI

Typed, validated and reliable LLM applications: from schemas and structured outputs to tool calling, agents and RAG.

```
class ToolCall(BaseModel):  
    name: Literal["search", "calculate"]  
    args: dict[str, Any]  
  
call = ToolCall.model_validate_json(llm_output)
```

Validation

Structured outputs

Tool calling

PydanticAI

Agents

RAG

Contents

01	Why Pydantic Matters in GenAI	3
	The typed boundary, Pydantic's five roles in AI stacks, and where it already lives in your tools	
02	Pydantic v2 Fundamentals	5
	BaseModel, Field, types, lax versus strict, configuration, nesting and JSON Schema	
03	Validators and Custom Logic	8
	Field and model validators, reusable Annotated validators, discriminated unions, error anatomy, context	
04	Structured Outputs from LLMs	12
	Prompt-and-parse, tool calling, native structured outputs, retry loops, schema design, streaming	
05	Tool Calling with Pydantic	15
	Tool anatomy, a schema-generating registry, validate_call, the tool loop, and tool design rules	
06	PydanticAI: Agents Built on Types	17
	Agent, output_type, dependencies, tools, output validators, ModelRetry, testing and observability	
07	Pydantic Across the Ecosystem	20
	LangChain and LangGraph, LlamaIndex, OpenAI Agents SDK, CrewAI, Instructor, DSPy, and a selection guide	
08	Designing Agentic Systems with Types	22
	Five validation touchpoints, typed actions, plans, state, agent handoffs, guardrails and memory	
09	RAG with Pydantic	25
	Chunk and metadata models, query analysis, grounded answers with checked citations, agentic RAG routing	
10	Settings, Config and Observability	27
	pydantic-settings, SecretStr, typed agent config, safe logging, tracing and schema versioning	
11	Production Playbook	29
	What goes wrong, how to harden it, and what to verify before you ship	
12	Cheat Sheet and Resources	31
	Quick API reference, LLM integration map and where to read more	

ABOUT THIS GUIDE

Code targets **Pydantic v2** and Python 3.10+. Agent and provider SDKs move fast: treat every third-party snippet as a pattern, and confirm exact signatures against the current docs before shipping. Model names in examples are placeholders for whichever model you use.

CHAPTER 01

Why Pydantic Matters in GenAI

01

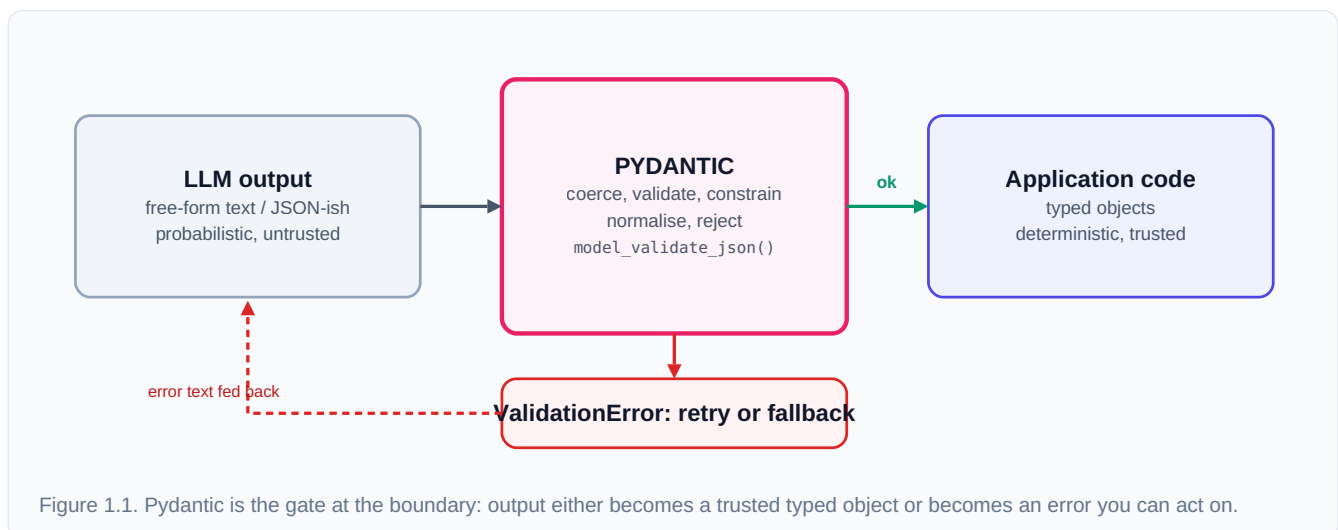
LLMs speak in text. Your software speaks in types. Pydantic is the contract that sits between them.

IN BRIEF

- The boundary between model output and application code is where reliability is won or lost.
- A Pydantic model gives you **schema, validation, serialization and documentation** from one declaration.
- Validation failures are not just errors: they are **feedback signals** a model can use to correct itself.

The core problem: probabilistic text meets deterministic code

A language model produces a sequence of tokens. Your application needs a `float` it can add, an enum it can branch on, a list it can loop over. Every serious LLM application has a **boundary** where free-form, probabilistic output crosses into deterministic code. Whatever happens at that boundary decides whether your system is a demo or a product.



Without a gate, the same four failures show up again and again:

Failure	What the model returned	What breaks downstream
Wrapped output	<code>```json {...}```</code> or a friendly sentence before the JSON	<code>json.loads</code> raises, request fails
Wrong type	<code>"total": "1,299.00"</code> instead of a number	Arithmetic fails, or worse, silently concatenates strings
Invented value	<code>"priority": "super-urgent"</code> when only low, medium, high exist	Routing logic falls through to a default branch
Missing or extra field	<code>due_date</code> omitted, or a made-up <code>confidence_note</code> added	<code>KeyError</code> in production, or unvalidated data persisted

Before and after

validation_before_after.py

```
# Without a contract: every consumer re-checks everything
data = json.loads(raw)           # may raise
total = float(data["total"])      # may KeyError / ValueError
if data.get("priority") not in {"low", "medium", "high"}:
    ...                           # and so on, in every call site

# With a contract: one declaration, one validation point
class Ticket(BaseModel):
    total: float = Field(ge=0)
    priority: Literal["low", "medium", "high"]

ticket = Ticket.model_validate_json(raw) # typed, constrained, or ValidationError
```

Pydantic's five roles in an AI stack

Role	What it does	Typical API
1. Schema	One class defines the shape of data and generates a JSON Schema from it	<code>Model.model_json_schema()</code>
2. Validation	Parses and checks model output, tool arguments, retrieved metadata	<code>model_validate_json()</code> , <code>TypeAdapter</code>
3. Serialization	Dumps typed objects to dicts or JSON for prompts, logs, APIs and storage	<code>model_dump(mode="json")</code>
4. Documentation	Field descriptions travel inside the schema, so they become instructions the model reads	<code>Field(description=...)</code>
5. Settings	Typed, validated configuration, API keys and agent limits from env vars or files	<code>pydantic-settings</code>

MENTAL MODEL

Think of a Pydantic model as a **contract with three audiences**: the model (reads the schema and descriptions), your code (gets typed attributes), and your future self (gets one place to see what the system promises).

Where you already meet Pydantic

Tool	How Pydantic shows up
OpenAI and Anthropic Python SDKs	Response objects are Pydantic models; OpenAI's <code>parse</code> helpers accept your own models as the output type
LangChain / LangGraph	Structured output, tool argument schemas and graph state can all be Pydantic models
LlamaIndex, CrewAI, DSPy	Typed outputs and task results declared with Pydantic models
PydanticAI, Instructor	Built around Pydantic: the output type is your model, validation errors drive retries
FastAPI and Python MCP servers	Request bodies and tool input schemas are derived from type hints

CHAPTER 02

Pydantic v2 Fundamentals

02

Everything you need from the core library, with the AI-specific angle on each feature.

IN BRIEF

- Use `model_validate_json` for raw model output and `model_json_schema` for the schema you send.
- `Literal` and `Field` constraints turn vague outputs into closed, checkable ones.
- Set `extra="forbid"` on tool arguments, and remember that `X | None` alone is still required.

BaseModel in one screen

basics.py

```

from datetime import date
from pydantic import BaseModel, Field

class Invoice(BaseModel):
    """An invoice extracted from a document."""
    vendor: str
    total: float = Field(ge=0, description="Grand total in USD")
    issued: date
    tags: list[str] = Field(default_factory=list)

inv = Invoice(vendor="Acme", total="129.50", issued="2026-10-01")
inv.total      # 129.5    (str coerced to float)
inv.issued    # datetime.date(2026, 10, 1)
inv.model_dump() # plain dict, ready for json.dumps

```

Types are enforced when the model is built. By default Pydantic runs in **lax mode**, which coerces sensible inputs (a numeric string to a float, an ISO string to a date). That is usually what you want for LLM output, which is rarely type-perfect.

The methods you will use daily

Method	Use it to
<code>Model.model_validate(obj)</code>	Validate a Python dict or object
<code>Model.model_validate_json(s)</code>	Validate a JSON string directly. Faster and stricter about JSON than <code>json.loads</code> first
<code>m.model_dump(mode="json")</code>	Get a JSON-safe dict (dates become strings, enums become values)
<code>m.model_dump_json(indent=2)</code>	Serialize straight to a JSON string for prompts and logs
<code>Model.model_json_schema()</code>	Generate the JSON Schema to hand to an LLM provider
<code>m.model_copy(update={...})</code>	Create a modified copy. Note: the update values are not re-validated
<code>Model.model_construct(...)</code>	Build without validation. Avoid on LLM output

Field: constraints and descriptions

Option	Meaning	Applies to
<code>gt, ge, lt, le</code>	Numeric bounds	int, float, Decimal
<code>min_length, max_length</code>	Length bounds	str, list, set, dict
<code>pattern</code>	Regular expression the string must match	str
<code>description, examples</code>	Documentation that lands in the JSON Schema and thus in the prompt	any
<code>default, default_factory</code>	Default value or a callable that builds one	any
<code>alias</code>	Different key name in the input data	any
<code>exclude</code>	Leave out of <code>model_dump</code> output	any

V2 GOTCHA: OPTIONAL IS NOT THE SAME AS OPTIONAL

`x: int | None` means *the value may be None*, but the field is still **required**. To make it omissible write `x: int | None = None`. This matters for LLM schemas: a required-but-nullable field forces the model to emit an explicit `null`.

Types worth knowing for AI work

Type	Why it helps with LLM output
<code>Literal["a", "b"]</code> , <code>Enum</code>	Closed sets. The schema lists the exact allowed values, so the model rarely invents new ones
<code>Annotated[T, Field(...)]</code>	Reusable constrained types, such as <code>Score = Annotated[float, Field(ge=0, le=1)]</code>
<code>StringConstraints</code>	Strip whitespace, enforce length or case in one declaration
<code>HttpUrl</code> , <code>EmailStr</code>	Validated URLs and emails (<code>EmailStr</code> needs the <code>email</code> extra)
<code>SecretStr</code>	Masks API keys in logs and reprs; read with <code>.get_secret_value()</code>
<code>Json[T]</code>	A string field that holds JSON, parsed and validated into <code>T</code>
<code>TypeAdapter(T)</code>	Validate any type (a list, a union) without defining a model class

Lax versus strict

Input	Lax (default)	Strict
<code>"42" to int</code>	42	ValidationError
<code>"true" to bool</code>	True	ValidationError
<code>"2026-10-01" to date</code>	date object	Accepted when parsing JSON, rejected from Python str
<code>3.0 to int</code>	3	ValidationError

Enable strict mode per model with `ConfigDict(strict=True)` or per field with `Field(strict=True)`. For LLM output, lax is normally the pragmatic choice. Reserve strict for places where silent coercion would hide a real bug, such as IDs and money.

Configuration that matters for agents

config.py

```
from pydantic import BaseModel, ConfigDict

class ToolArgs(BaseModel):
    model_config = ConfigDict(
        extra="forbid",          # reject unexpected keys (default is "ignore")
        str_strip_whitespace=True, # trim every string
        frozen=True,             # immutable and hashable
        validate_assignment=True, # re-validate on attribute assignment
    )
```

`extra="forbid"` is the single most useful setting for tool arguments: a hallucinated parameter becomes a clear error instead of being silently dropped.

Nested models and JSON Schema

schema.py

```
class Address(BaseModel):
    city: str
    country: str = Field(description="ISO 3166 alpha-2 code, e.g. 'PT'")

class Person(BaseModel):
    """A person mentioned in the text."""
    name: str
    address: Address | None = None

Person.model_json_schema()
# {"$defs": {"Address": {...}}, "description": "A person mentioned in the text.",
# "properties": {"name": {...}, "address": {"anyOf": [{"$ref": "#/$defs/Address"}, {"type": "null"}], ...}},
# "required": ["name"], "title": "Person", "type": "object"}
```

DOCSTRINGS AND DESCRIPTIONS ARE PROMPT TEXT

The class docstring becomes the schema `description`, and each `Field(description=...)` becomes a property description. Models read these. Write them as instructions to the model, not as comments for developers.

CHAPTER 03

Validators and Custom Logic

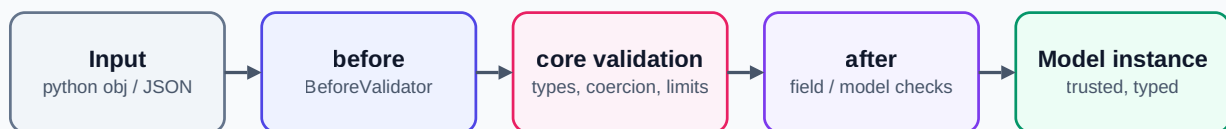
03

Go beyond types: clean up sloppy model output, enforce cross-field rules, and turn errors into feedback.

IN BRIEF

- Use **before** validators to tolerate messy output, **after** and model validators to enforce rules.
- Discriminated unions give agents a typed action space with precise errors.
- Format `ValidationError` into short, specific text: it is the retry prompt.
- Validation context checks output against facts only known at runtime.

How validation runs



wrap validators surround the whole sequence; plain validators replace the core step

Figure 3.1. Order of execution for a field. Validators can run before type coercion, after it, or wrap the whole step.

Mode	Receives	Typical use
<code>before</code>	Raw input, any type	Clean up messy model output: strip fences, parse "\$1,200", map "yes" to True
<code>after</code>	Value already coerced to the declared type	Business rules: ranges, relationships, normalisation
<code>wrap</code>	Raw input plus a handler to call the inner validation	Try inner validation, then recover on failure
<code>plain</code>	Raw input; replaces default validation entirely	Fully custom parsing

Field validators

field_validators.py

```

from pydantic import BaseModel, field_validator

class Review(BaseModel):
    rating: int
    summary: str

    @field_validator("rating", mode="before")
    @classmethod
    def parse_stars(cls, v):
        if isinstance(v, str) and v.endswith("/5"):  # model wrote "4/5"

```



```

        return int(v.split("/")[0])
    return v

    @field_validator("summary")
    @classmethod
    def not_blank(cls, v: str) -> str:
        if not v.strip():
            raise ValueError("summary must not be empty")
        return v.strip()

```

Reusable validators with Annotated

When the same rule appears in many models, attach it to the type instead of the class.

annotated_validators.py

```

from typing import Annotated
from pydantic import AfterValidator, BeforeValidator, Field

def to_float(v):
    return float(v.replace("$", "").replace(",", "")) if isinstance(v, str) else v

Money = Annotated[float, BeforeValidator(to_float), Field(ge=0)]
Score = Annotated[float, Field(ge=0.0, le=1.0)]
Slug = Annotated[str, AfterValidator(lambda s: s.lower().replace(" ", "-"))]

class Offer(BaseModel):
    price: Money          # "$1,299.00" becomes 1299.0
    relevance: Score
    tag: Slug

```

Cross-field rules with model_validator

model_validator.py

```

from typing_extensions import Self
from pydantic import model_validator

class LineItem(BaseModel):
    qty: int = Field(ge=1)
    unit_price: float = Field(ge=0)

class Order(BaseModel):
    items: list[LineItem]
    total: float

    @model_validator(mode="after")
    def total_matches(self) -> Self:
        expected = sum(i.qty * i.unit_price for i in self.items)
        if abs(expected - self.total) > 0.01:
            raise ValueError(f"total {self.total} does not match items sum {expected:.2f}")
        return self

```

ARITHMETIC IS WHERE MODELS SLIP

LLMs are unreliable at sums and counts. A cross-field validator catches the error and the retry message tells the model exactly which number to fix.

Computed fields

computed.py

```
from pydantic import computed_field

class Usage(BaseModel):
    input_tokens: int
    output_tokens: int

    @computed_field
    @property
    def total_tokens(self) -> int:
        return self.input_tokens + self.output_tokens
```

Computed fields appear in `model_dump()` and in the serialization schema, but the model is never asked to produce them. Let code derive what code can derive.

Discriminated unions: the backbone of agent actions

discriminated_union.py

```
from typing import Annotated, Any, Literal
from pydantic import BaseModel, Field, TypeAdapter

class SearchAction(BaseModel):
    type: Literal["search"]
    query: str

class CalcAction(BaseModel):
    type: Literal["calculate"]
    expression: str

class FinishAction(BaseModel):
    type: Literal["finish"]
    answer: str

Action = Annotated[SearchAction | CalcAction | FinishAction, Field(discriminator="type")]

action = TypeAdapter(Action).validate_json('{"type": "finish", "answer": "42"}')
# FinishAction(type='finish', answer='42')
```

The `type` field tells Pydantic which model to validate against, so errors point at the right branch rather than listing a failure for every union member. It also gives you clean `match` statements in your agent loop.

Reading ValidationError, and turning it into feedback

error_feedback.py

```
from pydantic import ValidationError

def format_errors(e: ValidationError) -> str:
    lines = []
    for err in e.errors(include_url=False, include_input=False):
        path = ".".join(str(p) for p in err["loc"]) or "(root)"
        lines.append(f"- {path}: {err['msg']}")
    return "\n".join(lines)

try:
    Order.model_validate_json(raw)
```

```
except ValidationError as e:
    feedback = format_errors(e)
    # - items.0.qty: Input should be greater than or equal to 1
    # - total: Value error, total 90.0 does not match items sum 120.00
```

Error key	Content
<code>loc</code>	Path to the failing field, such as <code>('items', 0, 'qty')</code>
<code>type</code>	Machine-readable code, such as <code>greater_than_equal</code> or <code>value_error</code>
<code>msg</code>	Human-readable message, which is also good model-readable feedback
<code>input</code>	The offending value. Drop it from logs when it may contain personal data

Validation context

```
context.py

from pydantic import ValidationInfo, field_validator

class Pick(BaseModel):
    doc_id: str

    @field_validator("doc_id")
    @classmethod
    def must_exist(cls, v: str, info: ValidationInfo) -> str:
        if v not in info.context["known_ids"]:
            raise ValueError(f"unknown doc_id {v!r}")
        return v

Pick.model_validate_json(raw, context={"known_ids": {"a1", "b2"}})
```

Context lets a validator check model output against **runtime facts**: retrieved chunk ids, the user's permitted resources, the current tool list. Chapter 9 uses this for citation checking.

CHAPTER 04

Structured Outputs from LLMs

04

Three ways to get typed data out of a model, how to design schemas it can follow, and how to recover when it cannot.

IN BRIEF

- Tool calling with a forced `tool_choice` is the most portable structured-output channel.
- Native structured outputs give valid JSON, but only Pydantic validation gives **correct-for-your-app** data.
- Schema design is prompt design: descriptions, closed sets, reasoning-first, shallow shapes.
- On failure, feed the formatted `ValidationError` back and retry a bounded number of times.

Three mechanisms

Mechanism	How it works	Strength	Weakness
Prompt and parse	Describe the format in the prompt, parse the text reply	Works with any model	Most failures; needs fence stripping and retries
Tool calling	Pass your schema as a tool's input schema; the model "calls" it with arguments	Widely supported, good adherence	Model may skip the tool unless you force it
Native structured outputs	Provider constrains decoding to your JSON Schema	Syntactically valid JSON by design	Supports only a subset of JSON Schema; not all models

GUARANTEED JSON IS NOT GUARANTEED CORRECTNESS

Even constrained decoding only guarantees the output *matches the schema*. It does not guarantee the values are true, within your business limits, or consistent across fields. Always validate with the full Pydantic model on your side.

PROVIDER SCHEMA LIMITS

Native modes typically accept a subset of JSON Schema. Some ignore or reject numeric and string constraints, restrict recursion and unions, or require every property to be listed as required with `additionalProperties: false`. Design the Pydantic model for the provider, then re-check constraints locally.

Pattern A: tool calling with the Anthropic SDK

anthropic_tool_output.py

```
import anthropic
from pydantic import BaseModel, Field

class Invoice(BaseModel):
    """Fields extracted from an invoice."""
    vendor: str
    total: float = Field(ge=0, description="Grand total in USD")

client = anthropic.Anthropic()
tool = {
    "name": "record_invoice",
```

```

    "description": "Record the invoice fields extracted from the text.",
    "input_schema": Invoice.model_json_schema(),
}
resp = client.messages.create(
    model="claude-sonnet-5-5", max_tokens=1024,
    tools=[tool], tool_choice={"type": "tool", "name": "record_invoice"},
    messages=[{"role": "user", "content": f"Extract the invoice:\n{text}"}],
)
block = next(b for b in resp.content if b.type == "tool_use")
invoice = Invoice.model_validate(block.input)  # your guarantee, not the provider's

```

`tool_choice` forces the model to answer through the tool, which is what turns tool calling into a structured-output channel.

Pattern B: native parsing with the OpenAI SDK

`openai_parse.py`

```

from openai import OpenAI

client = OpenAI()
completion = client.chat.completions.parse(
    model="gpt-4o",
    messages=[{"role": "user", "content": f"Extract the invoice:\n{text}"}],
    response_format=Invoice,  # a Pydantic class
)
invoice = completion.choices[0].message.parsed  # Invoice instance (or None on refusal)

```

Pattern C: Instructor, a thin retry layer

`instructor_example.py`

```

import instructor

client = instructor.from_provider("openai/gpt-4o-mini")
invoice = client.chat.completions.create(
    response_model=Invoice,
    max_retries=3,  # re-asks with the validation error text
    messages=[{"role": "user", "content": f"Extract the invoice:\n{text}"}],
)

```

Pattern D: a provider-agnostic validate-and-retry loop

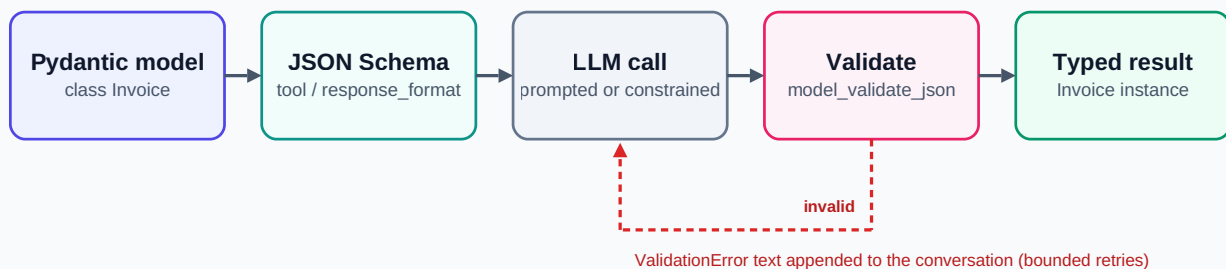


Figure 4.1. The reliable shape of structured output: schema out, validation in, and errors fed back to the model.

retry_loop.py

```
import re
from pydantic import BaseModel, ValidationError

def strip_fences(s: str) -> str:
    m = re.search(r"```(?:json)?\s*(.*?)```", s, re.S)
    return (m.group(1) if m else s).strip()

def extract(model_cls: type[BaseModel], prompt: str, call_llm, max_retries: int = 3):
    messages = [{"role": "user", "content": prompt}]
    for _ in range(max_retries + 1):
        raw = call_llm(messages)  # returns text
        try:
            return model_cls.model_validate_json(strip_fences(raw))
        except ValidationError as e:
            messages += [
                {"role": "assistant", "content": raw},
                {"role": "user", "content": "That JSON failed validation:\n"
                    + format_errors(e) + "\nReturn only the corrected JSON."},
            ]
    raise RuntimeError(f"No valid {model_cls.__name__} after {max_retries + 1} attempts")
```

Designing schemas models can follow

1. **Descriptions are prompts.** Say what to put in a field and how to handle edge cases, with an example value.
2. **Close the set.** Use `Literal` or `Enum` for categories rather than `str`.
3. **Offer an exit.** Add `"unknown"` or allow `None` so the model is not forced to guess.
4. **Reason first.** Put a short `reasoning` field before the answer fields; output is generated in order.
5. **Stay shallow.** Flat, small models are followed more reliably than deep nesting. Split big extractions into several calls.
6. **Bound and name well.** Use `max_length` and list limits; `due_date` beats `d1`, since names are part of the prompt.

schema_as_prompt.py

```
class Ticket(BaseModel):
    """A support ticket triaged from a customer message."""
    reasoning: str = Field(description="One or two sentences justifying the category. Write this first.")
    category: Literal["billing", "bug", "feature_request", "other"]
    priority: Literal["low", "medium", "high"] = Field(
        description="high only if the customer reports data loss or a full outage")
    summary: str = Field(max_length=200, description="Neutral one-line summary, no customer names")
    needs_human: bool
```

Streaming partial output

partial_streaming.py

```
from pydantic import TypeAdapter

ta = TypeAdapter(Ticket)
buffer = ""
for chunk in stream:  # text deltas from the model
    buffer += chunk
    partial = ta.validate_json(buffer, experimental_allow_partial=True)
    render(partial)  # update the UI with whatever is valid so far
```

PARTIAL VALIDATION IS EXPERIMENTAL

`experimental_allow_partial` exists in recent Pydantic v2 releases and may change. Fields required by the schema may be missing in partial results, so make the UI tolerant.

CHAPTER 05

Tool Calling with Pydantic

05

Turn Python functions into tools a model can call safely, with arguments you validate and errors the model can learn from.

IN BRIEF

- One Pydantic model per tool gives you schema and validation from a single source of truth.
- Return `ValidationError` text as the tool result so the model can self-correct.
- Bound the loop, narrow the types, and require approval for anything destructive.

Anatomy of a tool

Every provider describes a tool with three things: a **name**, a **description** that tells the model when to use it, and a **parameters schema**. Pydantic generates the third from a model and keeps validation and schema in sync, so they cannot drift apart.

tool_args.py

```
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field

class WeatherArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")
    city: str = Field(min_length=1, description="City name, e.g. 'Lisbon'")
    unit: Literal["c", "f"] = Field("c", description="Temperature unit")
```

A small registry that generates schemas and validates calls

tool_registry.py

```
import inspect
from typing import Callable, get_type_hints
from pydantic import BaseModel, ValidationError

REGISTRY: dict[str, dict] = {}

def tool(fn: Callable):
    hints = get_type_hints(fn)
    args_model = next(t for name, t in hints.items() if name != "return")
    REGISTRY[fn.__name__] = {
        "fn": fn,
        "args": args_model,
        "spec": {"name": fn.__name__,
                 "description": inspect.getdoc(fn) or "",
                 "input_schema": args_model.model_json_schema()},
    }
    return fn

def dispatch(name: str, raw_args: dict) -> str:
    entry = REGISTRY.get(name)
    if entry is None:
        return f"Error: unknown tool {name!r}. Available: {sorted(REGISTRY)}"
    try:
        args = entry["args"].model_validate(raw_args)
    except ValidationError as e:
```

```

        return "Error: invalid arguments.\n" + format_errors(e)
    return str(entry["fn"](args))

@tool
def get_weather(args: WeatherArgs) -> dict:
    """Get the current weather for a city. Use for any question about present conditions."""
    return {"city": args.city, "temp": 21, "unit": args.unit}

```

ERRORS ARE TOOL RESULTS

Return validation errors as the tool's result text instead of raising. The model reads them on its next turn and usually fixes the call, which is cheaper and more robust than crashing the run.

Plain functions with `validate_call`

```

validate_call.py

from pydantic import validate_call

@validate_call
def add(a: int, b: int = 0) -> int:
    return a + b

add("2", 3)          # 5, "2" coerced to int
add("two", 3)         # ValidationError, with a precise message

```

The tool-use loop

```

tool_loop.py

messages = [{"role": "user", "content": question}]
tools = [t["spec"] for t in REGISTRY.values()]

for _ in range(MAX_STEPS):
    # always bound the loop
    resp = client.messages.create(model=MODEL, max_tokens=1024, tools=tools, messages=messages)
    if resp.stop_reason != "tool_use":
        break
    # the model produced its final answer
    messages.append({"role": "assistant", "content": resp.content})
    results = [
        {"type": "tool_result", "tool_use_id": b.id, "content": dispatch(b.name, b.input)}
        for b in resp.content if b.type == "tool_use"
    ]
    messages.append({"role": "user", "content": results})

```

Rules for good tools

Rule	Why
Name tools as verb_noun (<code>search_orders</code>)	Names are part of the prompt; vague names cause wrong picks
Say when to use and when not to use it	Most mis-calls come from overlapping descriptions
Narrow argument types (<code>Literal</code> , bounds) and use <code>extra="forbid"</code>	Fewer invalid calls, hallucinated parameters fail loudly
Return compact, typed results (<code>model_dump_json()</code>)	Large raw payloads waste context and distract the model
Make destructive tools require an explicit confirmation field or human approval	Validation proves shape, not permission
Keep tools idempotent where possible	Retries and repeated calls become safe

CHAPTER 06

PydanticAI: Agents Built on Types

06

The agent framework from the Pydantic team: typed outputs, typed dependencies, validated tools and retries out of the box.

IN BRIEF

- PydanticAI turns **output type, dependencies and tool signatures** into the agent's contract.
- `ModelRetry` and output validators move business rules into the retry loop.
- `TestModel` and `UsageLimits` make agents testable and bounded from day one.

Why a Pydantic-native agent framework

PydanticAI makes the types you already write the centre of the agent. The `Agent` is generic over its **dependencies** and its **output**, so a type checker can tell you what a run returns. Tool signatures become tool schemas, docstrings become descriptions, and validation failures trigger automatic retries.

Your first typed agent

pydanticai_hello.py

```
from pydantic import BaseModel, Field
from pydantic_ai import Agent

class CityInfo(BaseModel):
    city: str
    country: str
    population_estimate: int = Field(description="Approximate population")

agent = Agent(
    "anthropic:claude-sonnet-5-5",
    output_type=CityInfo,
    instructions="Answer with accurate, concise facts.",
)
result = agent.run_sync("Tell me about Lisbon")
print(result.output)  # CityInfo(city='Lisbon', country='Portugal', ...)
```

Core concepts

Concept	What it is
<code>Agent</code>	Container for model, instructions, tools, output type and retry settings
<code>output_type</code>	A Pydantic model, a primitive, or a list of types for a union of possible results
<code>deps_type</code> / <code>RunContext</code>	Typed dependencies (DB handles, user id, HTTP client) injected into tools and instructions
<code>@agent.tool</code> , <code>@agent.tool_plain</code>	Register a tool with or without access to <code>RunContext</code>
<code>@agent.output_validator</code>	Extra validation on the final output, may raise <code>ModelRetry</code>

Concept	What it is
<code>ModelRetry</code>	Raise from a tool or validator to send a message back to the model and try again
<code>run</code> , <code>run_sync</code> , <code>run_stream</code>	Async, sync and streaming execution
<code>UsageLimits</code>	Cap requests or tokens per run to stop runaway loops
<code>TestModel</code> , <code>FunctionModel</code>	Deterministic stand-ins for tests without network calls

Dependencies and tools

pydanticai_deps.py

```
from dataclasses import dataclass
from pydantic import BaseModel, Field
from pydantic_ai import Agent, RunContext

@dataclass
class Deps:
    db: "Database"
    customer_id: int

class SupportReply(BaseModel):
    advice: str = Field(description="Advice to show the customer")
    escalate: bool = Field(description="True if a human should take over")
    risk: int = Field(ge=0, le=10, description="Risk level of the request")

support = Agent("openai:gpt-4o", deps_type=Deps, output_type=SupportReply,
                instructions="You are a careful bank support agent.")

@support.tool
async def balance(ctx: RunContext[Deps], include_pending: bool = False) -> float:
    """Return the current balance of the customer being helped."""
    return await ctx.deps.db.balance(ctx.deps.customer_id, include_pending)

result = await support.run("What is my balance?", deps=Deps(db=db, customer_id=42))
```

Output validators and ModelRetry

pydanticai_validator.py

```
from pydantic_ai import ModelRetry

@support.output_validator
async def check_escalation(ctx: RunContext[Deps], out: SupportReply) -> SupportReply:
    if out.risk >= 8 and not out.escalate:
        raise ModelRetry("Risk is 8 or higher, so escalate must be true. Revise the reply.")
    return out
```

Pydantic validation of the schema and your own validator share one retry budget (`retries` on the agent). Tools can raise `ModelRetry` too, for example when a lookup finds no match and the model should try different arguments.

Several possible outputs

pydanticai_union.py

```
class Found(BaseModel):
    answer: str
    source_id: str

class NotFound(BaseModel):
    reason: str
```

```
agent = Agent("anthropic:claude-sonnet-5-5", output_type=[Found, NotFound])
out = agent.run_sync("Who signed the 2019 contract?").output
if isinstance(out, Found):
    ...
```

Passing a list of output types gives the agent a typed branch: success and failure are both first-class results, not exceptions or magic strings.

Testing without a network

pydanticai_test.py

```
from pydantic_ai.models.test import TestModel

def test_agent_returns_valid_city():
    with agent.override(model=TestModel()):
        result = agent.run_sync("anything")
        assert isinstance(result.output, CityInfo)
```

Observability and limits

pydanticai_observe.py

```
import logfire
from pydantic_ai.usage import UsageLimits

logfire.configure()
logfire.instrument_pydantic_ai()  # traces runs, tool calls and validation retries

result = agent.run_sync("...", usage_limits=UsageLimits(request_limit=6))
```

PIN YOUR VERSIONS

Agent frameworks evolve quickly. Argument names such as `output_type` replaced older ones, and streaming helpers have changed. Pin a version, read the changelog on upgrades, and keep a test with `TestModel` covering each agent.

CHAPTER 07

Pydantic Across the Ecosystem

07

The same model class travels between frameworks. Here is where it plugs in, and how to choose.

IN BRIEF

- Almost every Python AI framework accepts a Pydantic model as the output or argument type.
- Keep domain models in a framework-neutral module and reuse them everywhere.
- Match the tool to the shape of the work: extraction, tool agent, graph workflow or retrieval pipeline.

Where the hook is in each framework

Framework	Pydantic hook	Minimal usage
LangChain	Structured output, tool <code>args_schema</code>	<code>llm.with_structured_output(Model)</code>
LangGraph	Graph state as a <code>BaseModel</code>	<code>StateGraph(State)</code>
LlamaIndex	Typed output from an LLM or query engine	<code>llm.as_structured_llm(Model)</code>
OpenAI Agents SDK	Agent output type and function tool schemas	<code>Agent(output_type=Model), @function_tool</code>
CrewAI	Typed task results	<code>Task(..., output_pydantic=Model)</code>
Instructor	Patched client returns your model	<code>response_model=Model, Partial[Model], Iterable[Model]</code>
DSPy	Typed signature fields can be Pydantic models	Field annotations in a <code>Signature</code>
FastAPI / FastMCP	Request bodies and MCP tool schemas from type hints	Annotated function parameters

LangChain and LangGraph

`langchain_langgraph.py`

```

from langchain.chat_models import init_chat_model
from pydantic import BaseModel, Field

class Ticket(BaseModel):
    category: str
    priority: int = Field(ge=1, le=5)

llm = init_chat_model("anthropic:claude-sonnet-5-5")
triage = llm.with_structured_output(Ticket)
ticket = triage.invoke("Customer says checkout fails with a 500 error")  # Ticket instance

# LangGraph: typed, validated state
from langgraph.graph import StateGraph

class State(BaseModel):

```

```

question: str
ticket: Ticket | None = None

graph = StateGraph(State)

```

OpenAI Agents SDK

openai_agents.py

```

from agents import Agent, Runner, function_tool
from pydantic import BaseModel

class Answer(BaseModel):
    text: str
    confidence: float

@function_tool
def lookup(term: str) -> str:
    """Look up a term in the glossary."""
    return GLOSSARY.get(term, "not found")

agent = Agent(name="Researcher", instructions="Be concise.", tools=[lookup], output_type=Answer)
result = Runner.run_sync(agent, "Define RAG")
print(result.final_output)  # Answer instance

```

Choosing a path

Situation	Reasonable default
One-off extraction or classification	Native structured outputs or Instructor with your model
Agent with tools, dependencies and tests	PydanticAI, or the agent SDK of your model vendor
Multi-step workflows with branching, persistence, human review	A graph framework such as LangGraph, with Pydantic state
Retrieval-heavy pipelines	LlamaIndex or LangChain components with Pydantic outputs
Exposing capabilities to other apps	An MCP server whose tool schemas derive from your models

KEEP MODELS FRAMEWORK-NEUTRAL

Define domain models (`Invoice` , `Ticket` , `Plan`) in your own package and import them into whichever framework you use. Then switching or mixing frameworks costs adapters, not rewrites.

CHAPTER 08

Designing Agentic Systems with Types

08

Agents are loops of decisions. Types make each decision inspectable, each handoff checkable and each failure recoverable.

IN BRIEF

- Treat the agent loop as five boundaries and validate at each one.
- A **discriminated union** of actions plus a **validated plan** make behaviour inspectable and testable.
- Typed state, handoffs and memory records stop small errors from compounding across steps.
- Risk-tag your tools and gate anything beyond read-only behind approval.

Five touchpoints for validation

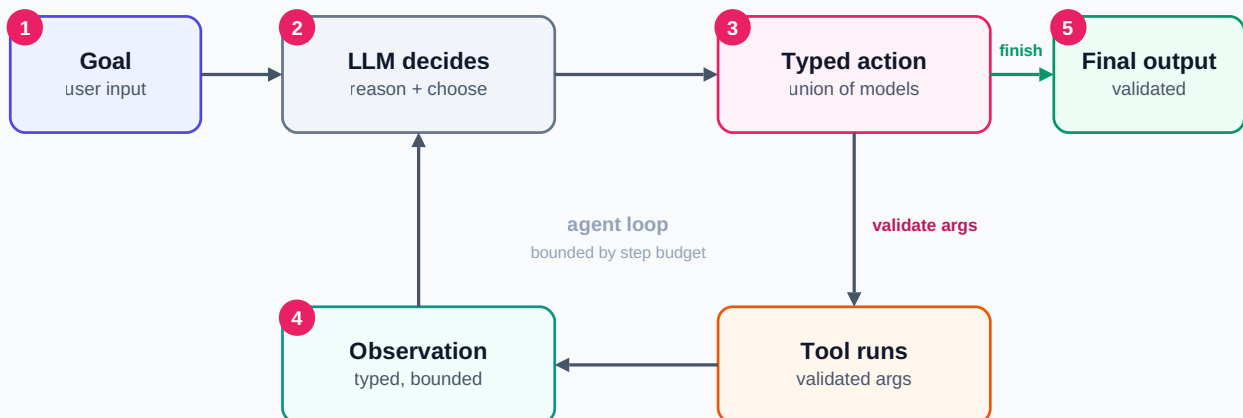


Figure 8.1. Where Pydantic sits in an agent loop.

#	Touchpoint	What to validate
1	Input	User goal and context: lengths, allowed formats, tenant and permission fields
2	Schemas out	Tool and output schemas generated from models, so the model sees exactly what is allowed
3	Actions and arguments	Each step the model proposes, parsed as a discriminated union with strict arguments
4	Observations	Tool results parsed into typed, size-bounded objects before re-entering the context
5	Final output	The answer, with business-rule validators and context checks

Pattern 1: a typed action space

action_space.py

```
from pydantic import TypeAdapter

action_ta = TypeAdapter(Action)  # the discriminated union from Chapter 3

def step(raw: str, state: "AgentState"):
    action = action_ta.validate_json(raw)
    match action:
        case SearchAction(query=q):
            state.observations.append(run_search(q))
        case CalcAction(expression=expr):
            state.observations.append(safe_eval(expr))
        case FinishAction(answer=a):
            return a
```

Pattern 2: plan-and-execute with a validated plan

plan.py

```
from typing_extensions import Self
from pydantic import BaseModel, Field, model_validator

class Step(BaseModel):
    id: str
    goal: str = Field(description="What this step must achieve")
    tool: Literal["search", "calculate", "write"]
    depends_on: list[str] = Field(default_factory=list)

class Plan(BaseModel):
    steps: list[Step] = Field(min_length=1, max_length=8)

    @model_validator(mode="after")
    def check_graph(self) -> Self:
        ids = [s.id for s in self.steps]
        if len(ids) != len(set(ids)):
            raise ValueError("step ids must be unique")
        for s in self.steps:
            missing = set(s.depends_on) - set(ids)
            if missing:
                raise ValueError(f"step {s.id} depends on unknown steps {sorted(missing)}")
        return self
```

A plan that references steps that do not exist is caught before any tool runs, and the error tells the planner exactly what to repair.

Pattern 3: typed agent state

agent_state.py

```
class AgentState(BaseModel):
    model_config = ConfigDict(validate_assignment=True)

    goal: str
    steps_left: int = Field(ge=0, le=25)
    observations: list[str] = Field(default_factory=list, max_length=50)
    done: bool = False

state = AgentState(goal="Summarise Q3 churn", steps_left=10)
state.steps_left -= 1  # re-validated on assignment; going below 0 raises
```

Pattern 4: contracts between agents

handoff.py

```
class Handoff(BaseModel):
    model_config = ConfigDict(extra="forbid")
    from_agent: str
    to_agent: Literal["researcher", "writer", "reviewer"]
    task: str = Field(max_length=500)
    context_ids: list[str] = Field(default_factory=list)
    expected_output: Literal["ResearchNotes", "Draft", "ReviewReport"]
```

In multi-agent systems the common bug is a vague message from one agent that the next one misreads. Give every edge an input model and an output model, and mismatches fail at the boundary rather than three steps later.

Pattern 5: guardrails and human approval

guardrail.py

```
class ToolRisk(str, Enum):
    read = "read"
    write = "write"
    destructive = "destructive"

class ApprovalRequest(BaseModel):
    tool: str
    risk: ToolRisk
    arguments: dict[str, Any]
    reason: str

def gate(tool: str, args: BaseModel, risk: ToolRisk, reason: str):
    if risk is ToolRisk.read:
        return "allow"
    return ApprovalRequest(tool=tool, risk=risk, arguments=args.model_dump(mode="json"), reason=reason)
```

Pattern 6: memory records

memory_item.py

```
class MemoryItem(BaseModel):
    schema_version: Literal[1] = 1
    text: str = Field(max_length=500)
    source: Literal["user", "tool", "model"]
    confidence: float = Field(ge=0, le=1)
    created_at: datetime
    expires_at: datetime | None = None
```

VALIDATE BOUNDARIES, NOT EVERY INTERNAL STEP

Put validation where data crosses a trust or ownership line: model to code, tool to model, agent to agent, storage to memory. Internal helper functions between those points do not need their own models.

CHAPTER 09

RAG with Pydantic

09

Retrieval pipelines pass many small structured objects. Typing them improves quality, debuggability and trust.

IN BRIEF

- Type every object that crosses a RAG stage: metadata, chunks, plans, grades and answers.
- Use validation **context** to check citations against the chunks actually retrieved.
- Typed routing decisions make agentic RAG branches explicit and bounded.

A typed pipeline

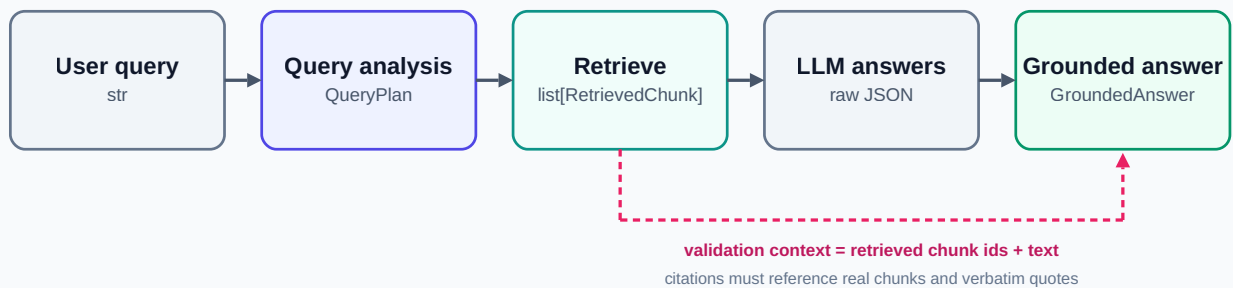


Figure 9.1. Each stage exchanges a typed object. The final answer is validated against what was actually retrieved.

Stage	Model	What validation buys you
Ingestion	<code>DocumentMeta</code>	Bad dates, empty text and malformed URLs never reach the index
Query analysis	<code>QueryPlan</code>	Bounded sub-queries and valid filters before retrieval runs
Retrieval	<code>RetrievedChunk</code>	Consistent ids, scores and source fields for downstream code
Generation	<code>GroundedAnswer</code>	Citations that must exist and quote real text

Core models

```

rag_models.py

from datetime import datetime
from pydantic import BaseModel, Field, HttpUrl

class DocumentMeta(BaseModel):
    source_url: HttpUrl
    title: str = Field(min_length=1)
    doc_type: Literal["paper", "blog", "docs", "report"]

class Chunk(BaseModel):
    id: str
    text: str = Field(min_length=1)
  
```

```

meta: DocumentMeta

class RetrievedChunk(BaseModel):
    chunk: Chunk
    score: float

class Filters(BaseModel):
    year_from: int | None = None

class QueryPlan(BaseModel):
    needs_retrieval: bool
    sub_queries: list[str] = Field(min_length=1, max_length=4)
    filters: Filters = Field(default_factory=Filters)

```

Grounded answers with checked citations

grounded_answer.py

```

from pydantic import ValidationInfo, field_validator

class Citation(BaseModel):
    chunk_id: str
    quote: str = Field(max_length=300, description="Exact words copied from the chunk")

class GroundedAnswer(BaseModel):
    answer: str
    citations: list[Citation] = Field(min_length=1)
    confidence: Literal["high", "medium", "low"]

    @field_validator("citations")
    @classmethod
    def citations_are_real(cls, cites: list[Citation], info: ValidationInfo):
        chunks: dict[str, str] = info.context["chunks"]  # id -> text
        for c in cites:
            if c.chunk_id not in chunks:
                raise ValueError(f"citation {c.chunk_id!r} was not retrieved")
            if c.quote not in chunks[c.chunk_id]:
                raise ValueError(f"quote for {c.chunk_id!r} does not appear in that chunk")
        return cites

ctx = {"chunks": {r.chunk.id: r.chunk.text for r in retrieved}}
answer = GroundedAnswer.model_validate_json(raw, context=ctx)

```

This does not prove the answer is true, but it cheaply blocks citations to sources that were never retrieved and invented quotes.

Agentic RAG: typed decisions

A typed `Route` lets the pipeline retrieve, clarify or answer. Pair it with a grader model (a `relevant: bool` plus a short reason) and a **bounded** rewrite loop so it cannot retrieve forever.

agentic_rag.py

```

class Retrieve(BaseModel):
    type: Literal["retrieve"]; query: str

class Clarify(BaseModel):
    type: Literal["clarify"]; question: str

class Answer(BaseModel):
    type: Literal["answer"]; text: str

Route = Annotated[Retrieve | Clarify | Answer, Field(discriminator="type")]

```

CHAPTER 10

Settings, Config and Observability

10

Agents have many knobs. Validate them at startup, keep secrets out of logs and make behaviour traceable.

IN BRIEF

- Validate settings and agent config at startup, and bound steps, timeouts and retries.
- Use `SecretStr` and `include_input=False` to keep secrets and personal data out of logs.
- Track first-pass validation rate and per-field errors: they point straight at prompt and schema fixes.

Typed settings with pydantic-settings

settings.py

```
from pydantic import Field, SecretStr
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    anthropic_api_key: SecretStr
    model_name: str = "claude-sonnet-5-5"
    temperature: float = Field(0.0, ge=0, le=1)
    max_tool_steps: int = Field(8, ge=1, le=50)
    request_timeout_s: float = Field(30, gt=0)

settings = Settings()  # reads ANTHROPIC_API_KEY, MODEL_NAME, ... from the environment
key = settings.anthropic_api_key.get_secret_value()
```

A misconfigured agent should fail at startup with a clear message, not three hours into a run. Bounds on `max_tool_steps` and timeouts double as safety limits.

Agent configuration as data

agent_config.py

```
class AgentConfig(BaseModel):
    name: str
    instructions: str
    tools: list[str]
    model: str
    max_steps: int = Field(8, ge=1, le=50)

configs = [AgentConfig.model_validate(c) for c in yaml.safe_load(open("agents.yaml"))]
```

Logging without leaking

- Log `model_dump(mode="json")` for structured, searchable records.
- Use `SecretStr` for keys and tokens so reprs show asterisks.

- Log validation failures with `e.errors(include_input=False)` when inputs may contain personal data.
- Mark sensitive fields with `Field(exclude=True)` to keep them out of dumps.

Tracing and measuring

observability.py

```
import logfire

logfire.configure()
logfire.instrument_pydantic()  # records validation successes and failures per model
```

Metric	What it tells you
First-pass validation rate	How well the prompt and schema fit the model. Falling rates signal drift
Retries per call	Cost and latency overhead of self-correction
Errors by field	Which fields to simplify, describe better or constrain differently
Tool-call error rate	Whether tool descriptions and argument types are clear
Tokens and latency per run	Budget and user experience

Versioning schemas

schema_versioning.py

```
class NoteV2(BaseModel):
    schema_version: Literal[2] = 2
    text: str
    tags: list[str] = Field(default_factory=list)

    @model_validator(mode="before")
    @classmethod
    def upgrade_v1(cls, data):
        if isinstance(data, dict) and data.get("schema_version", 1) == 1:
            data = {**data, "schema_version": 2, "tags": [data["category"]] if "category" in data else []}
        return data
```

Stored memory, cached outputs and eval datasets outlive your code. A `schema_version` field and a `before` validator that upgrades old shapes keep them loadable.

CHAPTER 11

Production Playbook

11

Failure modes, performance, security, testing, anti-patterns and a launch checklist.

IN BRIEF

- Most production failures are valid-but-wrong, truncated or injected output: plan for each explicitly.
- Validation is a quality tool, not a security boundary; enforce authorization in your own code.
- Measure first-pass validity and retries, and test with recorded outputs and deterministic models.

Failure modes and mitigations

Failure	Symptom	Mitigation
Invalid JSON	Parse error, fenced or chatty output	Native structured output or forced tool call; fence stripping; bounded retry
Truncation	JSON cut off at <code>max_tokens</code>	Raise limits, shrink the schema, detect <code>stop_reason</code> of max tokens before parsing
Valid but wrong	Passes the schema, fails reality	Cross-field validators, context checks, evals on real data
Enum drift	New or near-miss category values	<code>Literal</code> sets plus an <code>"other"</code> option; before-validator mapping
Invented identifiers	IDs or citations that do not exist	Validation context against known ids
Refusal or empty reply	No tool call or null parsed object	Handle explicitly; fall back or ask a clarifying question
Retry storm	Cost and latency spike	Cap retries, track retry rate, alert on thresholds
Prompt injection through data	Tool args or output steered by retrieved text	Strict argument schemas, allowlists, approval for risky tools

Performance

- Build a `TypeAdapter` **once** at module level and reuse it; constructing one per call is slow.
- Prefer `model_validate_json` to `json.loads` followed by `model_validate`.
- Use discriminated unions rather than plain unions of many models; validation picks the branch directly.
- For very large or rarely used models, `defer_build` can reduce import time.
- Keep schemas small: fewer fields mean fewer tokens in every request and fewer places to fail.

Security

VALIDATION IS NOT AUTHORIZATION

A correctly shaped argument can still be dangerous. Treat all model output as **untrusted input**.

- Use `extra="forbid"`, length limits and patterns on every tool argument model.
- Allowlist paths, hosts, tables and commands. Never pass validated strings to `eval`, shells or raw SQL.

- Check permissions against the **user's** identity (from your own dependencies), never from fields the model supplies.
- Require human approval for destructive or irreversible actions.
- Never put secrets in schemas or prompts; keep them in `SecretStr` settings and inject them in code.

Testing

Layer	Test
Models	Golden valid and invalid examples per model, including validators and edge cases
Prompts and schemas	Recorded real model outputs replayed through validation, run in CI
Agents	Deterministic stand-in models (for example <code>TestModel</code>) to test loops, tools and retries
Quality	Eval sets scoring field-level accuracy, first-pass validity and retry rate
Regressions	Re-run evals on every prompt, schema or model change

Anti-patterns

Avoid	Do instead
One giant schema extracting everything at once	Several focused calls with small models
<code>Optional</code> on every field	Require what you need; allow <code>None</code> only where absence is meaningful
Free-form <code>str</code> for categories	<code>Literal</code> or <code>Enum</code>
Catching <code>ValidationError</code> and moving on	Feed it back, fall back explicitly, or fail loudly
Unbounded retries or loops	Retry and step budgets with alerts
<code>model_construct</code> on model output	<code>model_validate</code> or <code>model_validate_json</code>
<code>Any</code> and bare <code>dict</code> in schemas	Typed nested models
Describing the format twice, in the prompt and in the schema	One schema; keep prose for intent, not shape

Pre-launch checklist

- ☐ Every model-to-code boundary parses through a Pydantic model.
- ☐ Tool argument models use `extra="forbid"` with bounded types.
- ☐ Retries, steps, tokens and timeouts are capped and tested.
- ☐ Validation errors are formatted and fed back to the model.
- ☐ Citations and identifiers are checked against runtime context.
- ☐ Secrets use `SecretStr`; logs exclude sensitive inputs.
- ☐ Destructive tools need approval; permissions come from your code.
- ☐ Eval tracks first-pass validity, retry rate and field accuracy.
- ☐ Library versions are pinned and upgrades are tested.

CHAPTER 12

Cheat Sheet and Resources

12

The one-page reference you will keep open while building.

IN BRIEF

- Define once with Pydantic, send the schema to the model, validate the reply, feed errors back.
- Types at every boundary, bounds on every loop, approval on every destructive action.

Pydantic quick reference

Task	Code
Define a model	<code>class M(BaseModel): x: int = Field(ge=0, description="...")</code>
Validate JSON	<code>M.model_validate_json(raw)</code>
Validate a dict	<code>M.model_validate(data, context={...})</code>
Validate any type	<code>TypeAdapter(list[M]).validate_json(raw)</code>
Dump	<code>m.model_dump(mode="json")</code> , <code>m.model_dump_json()</code>
JSON Schema	<code>M.model_json_schema()</code>
Closed set	<code>Literal["a", "b"]</code> or an <code>Enum</code>
Reject extra keys	<code>model_config = ConfigDict(extra="forbid")</code>
Clean messy input	<code>@field_validator("x", mode="before")</code> , <code>BeforeValidator</code>
Cross-field rule	<code>@model_validator(mode="after")</code>
Union of actions	<code>Annotated[A B, Field(discriminator="type")]</code>
Runtime facts in validators	<code>info.context</code> with <code>context=</code> on validate
Error text	<code>e.errors(include_url=False, include_input=False)</code>
Secrets	<code>SecretStr</code> , <code>.get_secret_value()</code>
Settings	<code>pydantic_settings.BaseSettings</code>

LLM integration map

Goal	Reach for
Typed answer from any provider	Forced tool call with <code>model_json_schema()</code> , then <code>model_validate</code>
Native parsing	Provider helper that accepts a Pydantic class (for example OpenAI <code>parse</code>)
Retries with error feedback	Instructor, PydanticAI, or your own loop with <code>format_errors</code>
Tool definitions	One args model per tool, with <code>extra="forbid"</code>

Goal	Reach for
Typed agent with dependencies	PydanticAI <code>Agent(deps_type=..., output_type=...)</code>
Graph workflows	LangGraph with a <code>BaseModel</code> state
Grounded RAG answers	Citation validators using <code>context</code>
Configuration	<code>pydantic-settings</code> with <code>SecretStr</code>

Further reading

- **Pydantic documentation:** docs.pydantic.dev (models, validators, JSON Schema, TypeAdapter, performance tips)
- **PydanticAI:** ai.pydantic.dev (agents, tools, outputs, testing, evals)
- **Instructor:** python.useinstructor.com (structured outputs and retries across providers)
- **pydantic-settings:** docs.pydantic.dev, settings management section
- **JSON Schema:** json-schema.org (the vocabulary behind every schema you send)
- **Provider docs:** the structured outputs and tool use pages of your model vendor, for current supported schema features

A GOOD FIRST PROJECT

Take one LLM call in your codebase that currently parses text by hand. Define a model, add a forced tool call or native structured output, validate with `model_validate_json`, add the error-feedback retry loop, and track first-pass validity. That single change usually removes the most fragile code in the system.